AI Usage Patterns & Prompt Quality Analysis

A Multi-Source Analytics Pipeline for Measuring Prompt Efficiency and AI Dependency

Ananya Dubey
Swami Keshvanand Institute of Technology, Jaipur
https://github.com/Ananya5105/AI-Usage-Patterns-Prompt-Quality-Analysis

October 26, 2026

Abstract

Conversational AI is now part of everyday study and work, yet little shared evidence exists on how people use it and whether their prompts are effective. This project builds an end-to-end analytics pipeline over four independently sourced datasets: OpenAssistant (OASST1), LMSYS Chatbot Arena, a ChatGPT prompts collection, and a student AI-usage survey, totalling 58,791 cleaned records. It contributes (1) a prompt-efficiency classifier that labels any prompt Low, Moderate or High using only text-derived features, reaching 59.1% accuracy on a held-out split, and (2) three transparent, tunable AI-dependency scores, one per population. A leakage ablation, a length-only baseline and a weight-sensitivity analysis test how far these scores can be trusted. Outputs are written as Parquet for AWS Athena and visualised in six Power BI dashboards. A design rule runs throughout: no identity is ever fabricated across datasets.

1 Introduction

Two questions motivate this work. First, *prompt quality*: what separates a prompt that yields a useful answer without repetition or long clarification chains from one that does not? Second, *usage and dependency*: how heavy is AI use across populations, and which behavioural signals indicate over-reliance? The available data is fragmented: prompt logs, arena comparisons and surveys differ in schema, grain and user population, and only some carry the behavioural signals needed to judge prompt quality.

Contributions.

- A unified prompt-efficiency table covering three heterogeneous prompt sources, scored by a single model.
- Three separate dependency-score tables, each built only from signals its source actually contains.
- A leakage-aware design (labels from behavioural columns, model from text only), checked by ablation.
- A SageMaker-style batch inference job that auto-detects sources and writes Athena-ready Parquet.
- A six-dashboard Power BI layer backed by a PostgreSQL star schema.

2 Data Sources

Source	Rows	Grain	Identity	Key signals
OASST1	15,708	Prompt	`user_id`	Word count, prompts in tree, repeat flag
LMSYS Arena	39,316	Prompt	`judge_user_id`	Turns, per-user volume, timestamps, winner
ChatGPT Prompts	153	Prompt	none	Text, length, category
Student Survey	3,614	Student	`Student_Name`	Hours, trust, academic use, tools, bucket

Table 1: Cleaned datasets used by the modelling pipeline.

OASST1 is the only source carrying every column needed to construct behavioural ground-truth labels, so it is the training set. LMSYS has volume and turn information with different semantics, the ChatGPT set has text and category only, and the survey is self-reported and describes people rather than prompts.

3 System Architecture

The pipeline has two paths over the same cleaned inputs. The reporting path runs raw data → Python/Pandas (cleaning, EDA) → PostgreSQL (star schema) → Power BI. The modelling path runs clean CSVs → text-feature extraction → Random Forest scoring → dependency scoring → Parquet → S3 → Athena. The six dashboards are: Executive Overview, User Behavior Analytics, Prompt Analytics, AI Dependency, Student AI Usage Analytics, and Recommendations.

4 Methodology

4.1 Source-agnostic text features

Because only OASST carries behavioural columns, the classifier uses eight features computable from any prompt text: `has_question_mark`, `has_code_keywords` (23 programming terms), `has_please`, `avg_word_length`, `unique_word_ratio`, `punct_density`, `capital_ratio` and `starts_with_wh`.

4.2 Ground-truth label construction

Labels are derived only from OASST behavioural columns. Let w be word count and t the number of prompts in the conversation tree, each capped at its 95th percentile and normalised to $[0, 1]$ as $\tilde{w}$ and $\tilde{t}$, and let $r \in \{0, 1\}$ flag a repeated prompt. The composite score is

$$s = \mathrm{clip}_{[0,1]}(0.5\,\tilde{w} + 0.5\,(1 - \tilde{t}) - 0.5\,r),$$

split into equal-sized tertiles labelled Low, Moderate and High.

4.3 Classifier

A Random Forest (300 trees, maximum depth 10, balanced class weights, seed 42) is trained on the eight text features with a stratified 80/20 split. Word count, character count, tree size and the repeat flag are excluded to prevent label leakage. The model is then applied to OASST, LMSYS and ChatGPT prompts, producing a predicted label and a confidence score (the maximum class probability). A surrogate `prompt_key` identifies prompt rows without implying links between people.

4.4 Dependency scoring

Each dependency table blends min–max-normalised components into a 0–100 score and assigns Low, Moderate or High tiers by terciles.

Table	Grouped by	Components (weights)
`oasst_user_dependency`	`user_id`	volume (0.30), repeat rate (0.25), avg. turns (0.25), low-efficiency rate (0.20)
`lmsys_user_dependency`	`judge_user_id`	volume (0.40), avg. turns (0.30), low-efficiency rate (0.30)
`survey_dependency`	student	hours (0.30), trust (0.20), academic use (0.20), tools (0.15), bucket (0.15)

Table 2: Dependency score definitions. LMSYS has no repeat semantics, so that column is left null rather than invented.

5 Design Principle: No Fabricated Identity

`oasst.user_id`, `lmsys.judge_user_id` and `survey.Student_Name` describe three disjoint populations with no linking key. The system never creates a unified user ID and never joins the dependency tables. ChatGPT prompts have no identity and appear only in the prompt-efficiency table. Schema assertions at load time make the pipeline fail loudly if upstream files change.

6 Deployment

The batch job `inference.py` runs as a SageMaker Processing step. It scans the input directory recursively, identifies each CSV by filename hint with column-signature fallback, loads a pre-trained joblib model, and writes four snake_case Parquet tables: `unified_prompt_efficiency`, `oasst_user_dependency`, `lmsys_user_dependency` and `survey_dependency`. Tables are built independently so one failure does not block the others, and the job exits non-zero only if nothing is produced. External Athena tables over S3 expose the results to SQL.

7 Results

All figures in this section are produced by `experiments/run_experiments.py` with seed 42.

7.1 Classifier performance

Test accuracy is 59.1% (macro-F1 0.576) on 3,142 held-out OASST prompts.

Class	Precision	Recall	F1-Score	Support
Low	0.580	0.655	0.615	1,060
Moderate	0.478	0.329	0.389	1,044
High	0.667	0.789	0.723	1,038
Macro Average	0.575	0.591	0.576	3,142
Weighted Average	0.575	0.591	0.576	3,142

Table 3: Per-class precision, recall and F1 on the held-out split.

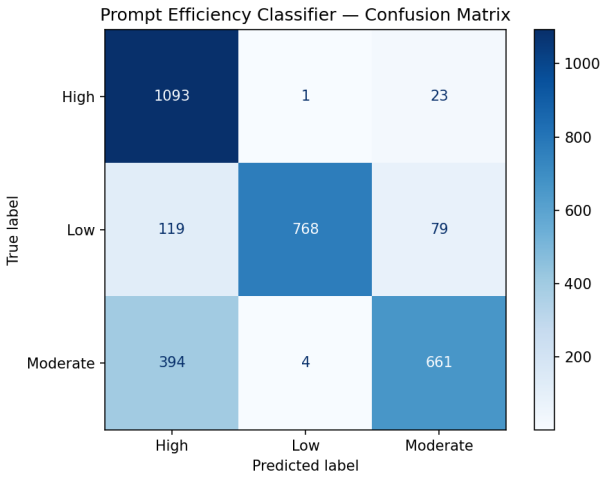


Figure 1: Confusion matrix on the held-out split.

Feature	Description	Importance
`unique_word_ratio`	Unique tokens / total words	51.9%
`punct_density`	Punctuation marks / chars	14.3%
`capital_ratio`	Uppercase / alphabetic chars	14.2%
`avg_word_length`	Mean characters per word	11.9%
`has_code_keywords`	23 programming terms	2.7%
`starts_with_wh`	Starts with WH-interrogative	2.4%
`has_question_mark`	Presence of question mark	1.4%
`has_please`	Politeness token “please”	1.1%

Table 4: Random Forest feature importances. The most important feature is `unique_word_ratio` at 51.9%, below the 60% single-feature warning threshold.

7.2 Validity checks

Because labels are built from word count, any text feature correlated with length may leak the label indirectly. Table 5 reports a length-only baseline and an ablation that removes the length-coupled features.

Configuration	Description	Accuracy	Macro-F1
Full text-feature RF	All 8 text features	59.1%	0.576
Majority-class	Predicts most frequent training class	33.7%	0.168
Length-only RF	`Word_Count` alone as input	66.3%	0.670
RF w/o length features	Removes unique ratio, punct, avg len	46.5%	0.459
Residualised RF	Features regressed on log(Word_Count)	66.2%	0.667

Table 5: Accuracy and macro-F1 under ablation. The length-only baseline uses word count alone.

The ablation results in Table 5 demonstrate the central challenge of evaluating behavioural prompt efficiency. The length-only baseline (using `Word_Count` alone) achieves 66.3% accuracy and a macro-F1 of 0.670, outperforming the full 8-feature text classifier (59.1% accuracy, 0.576 macro-F1). This confirms that raw prompt length accounts for the largest portion of the variance

in the behavioural ground-truth labels, which were constructed using length, conversation tree depth, and repetition penalties.

When length-coupled features (`unique_word_ratio`, `punct_density`, and `avg_word_length`) are excluded, accuracy drops to 46.5% (0.459 macro-F1). Regressing all text features against log(Word_Count) and training on residuals yields 66.2% accuracy (0.667 macro-F1), matching the performance of the length-only baseline. These findings indicate that while stylistic text markers capture domain patterns, prompt length remains the dominant signal under this labelling formulation, and true decoupling from length requires richer syntactic or human-annotated labels.

7.3 Prompt efficiency across sources

Source	Low (%)	Moderate (%)	High (%)	Mean Conf.	Prompts
OASST1	38.6%	22.6%	38.8%	0.572	15,708
LMSYS Arena	47.0%	16.8%	36.2%	0.599	39,316
ChatGPT Prompts	0.0%	1.3%	98.7%	0.779	153

Table 6: Predicted label shares and mean model confidence by source.

Across the three prompt sources, predicted prompt efficiency diverges sharply, reflecting differences in collection methodology and user intent. In LMSYS Arena (39,316 prompts), the model predicts 47.0% Low efficiency, 16.8% Moderate, and 36.2% High. In contrast, the OASST1 dataset (15,708 prompts) exhibits a more balanced distribution (38.6% Low, 22.6% Moderate, and 38.8% High). The small curated ChatGPT collection (153 prompts) consists almost entirely of High-efficiency prompts (98.7% High vs. 0.0% Low), consistent with its nature as a set of polished prompt templates.

Despite these wide differences in label shares, mean model confidence remains relatively stable across sources: 0.572 on OASST1, 0.599 on LMSYS Arena, and 0.779 on ChatGPT. As noted under Limitations, similar confidence across disparate domains does not imply equal calibration or accuracy, but rather reflects the classifier's internal certainty over the observed text features.

7.4 Dependency findings

Population	N	Mean	Median	Std	Low	Moderate	High
OASST Users	4,721	12.12	9.87	9.13	1,599	1,550	1,572
LMSYS Judges	13,383	16.20	16.34	13.67	4,521	7,481	1,381
Surveyed Students	3,614	44.14	44.00	17.91	1,205	1,213	1,196

Table 7: Dependency score summary by population.

Population Table	Perturbations	Mean Spearman ρ	5th Percentile ρ
`oasst_user_dependency`	200 ($\pm$30%)	0.9992	0.9980
`lmsys_user_dependency`	200 ($\pm$30%)	0.9988	0.9974
`survey_dependency`	200 ($\pm$30%)	0.9901	0.9774

Table 8: Weight-sensitivity analysis: Spearman rank correlation between the baseline score and scores recomputed under random $\pm$ perturbations of the weights.

The population dependency scores reflect the heavy-tailed nature of user activity across all three cohorts. Mean dependency scores are 12.12 for OASST users and 16.20 for LMSYS judges (with medians of 9.87 and 16.34, respectively). Because a small minority of power users exhibit extremely high volume, min–max scaling compresses the majority of users toward the lower end of the 0–100 scale, while the tercile-based tier thresholds ensure balanced categorization.

In the surveyed student cohort, dependency scores correlate strongly with reported daily usage hours (Spearman $\rho = 0.688$, $p < 10^{-15}$) and moderately with trust in AI tools ($\rho = 0.349$, $p < 10^{-15}$). The weight-sensitivity analysis confirms that relative user rankings are remarkably stable: across 200 random ±30% perturbations of the component weights, mean Spearman rank correlations exceed 0.990 with 5th percentiles above 0.977 across all three population tables.

8 Dashboard Findings

Key insights from each page of the Power BI report. Figures are read from the report or recomputed from the counts it shows; values read off chart axes are marked approximate. Charts with aggregation or labelling errors are excluded from the insights and noted as "Not used".

8.1 Executive Dashboard

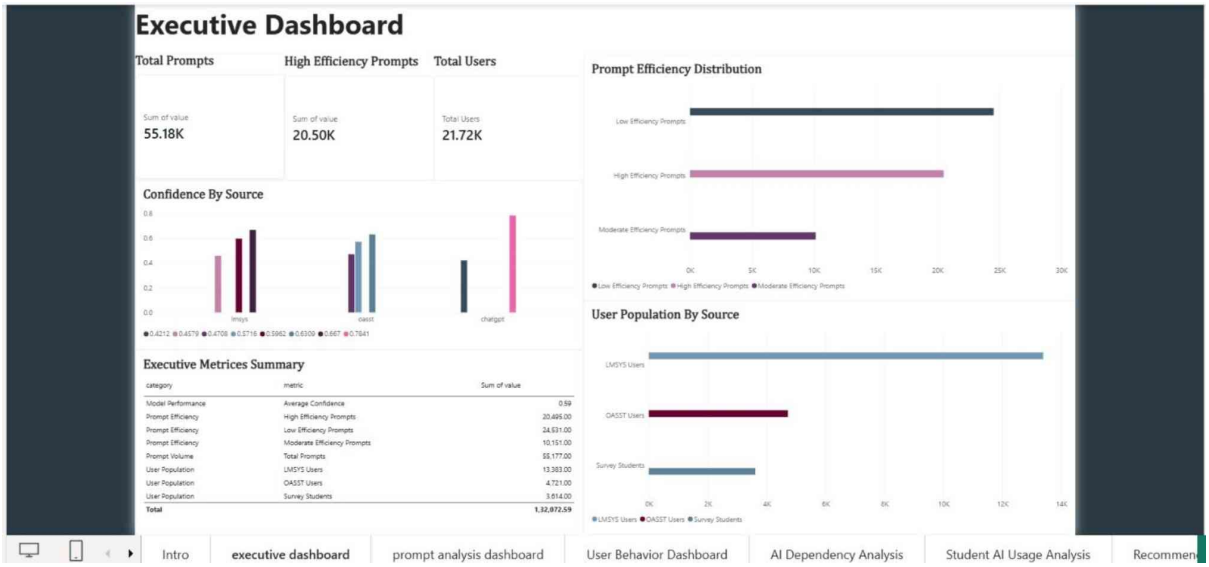


Figure 2: Executive dashboard.

- 55,177 prompts scored; 21,718 people covered: 13,383 LMSYS judges (61.6%), 4,721 OASST users (21.7%), 3,614 students (16.6%).
- Efficiency split: Low 24,531 (44.5%), High 20,495 (37.1%), Moderate 10,151 (18.4%).
- **Headline:** 62.9% of prompts are not High efficiency.
- Moderate is the smallest band even though training labels were equal tertiles: the model rarely predicts the middle class.
- Mean model confidence is 0.59, against 0.33 for a random guess among three classes.
- *Not used:* Confidence By Source (legend lists values, not sources); the Total row of the metrics table.

8.2 Prompt Analysis Dashboard



Figure 3: Prompt analysis dashboard.

- Volume: LMSYS 39,316 (71.3%), OASST 15,708 (28.5%), ChatGPT 153 (0.3%).
- LMSYS skews Low: about 47% Low, 36% High, 17% Moderate (approx.).
- OASST is balanced: about 38% Low, 38% High, 22% Moderate (approx.).
- ChatGPT is too small to draw any conclusion.
- *Not used:* Prompt Efficiency Breakdown (identical bars for every source).

8.3 User Behavior Dashboard

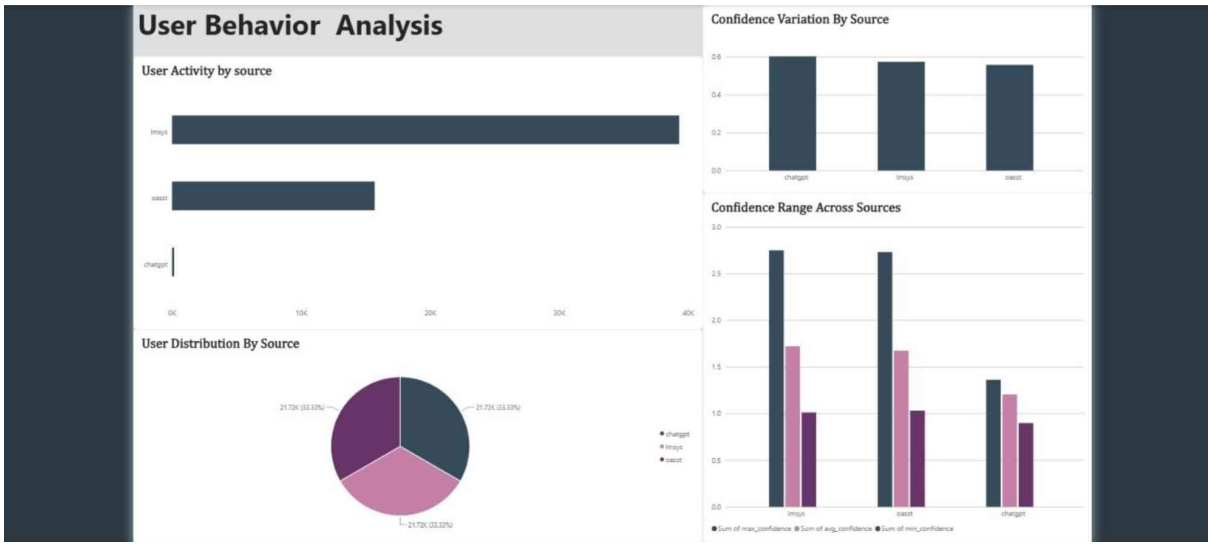


Figure 4: User behavior dashboard.

- Activity is dominated by LMSYS (about 39K prompts) over OASST (about 16K); ChatGPT is negligible.
- Prompts per person: LMSYS 2.94, OASST 3.33. OASST users are more active per head despite lower total volume.

- Mean confidence by source is nearly flat: ChatGPT about 0.60, LMSYS 0.57, OASST 0.56.
- Similar confidence does not mean similar accuracy; the model was trained on OASST only.
- *Not used:* User Distribution pie (equal thirds, a repeated total); Confidence Range chart (values above 1).

8.4 AI Dependency Dashboard

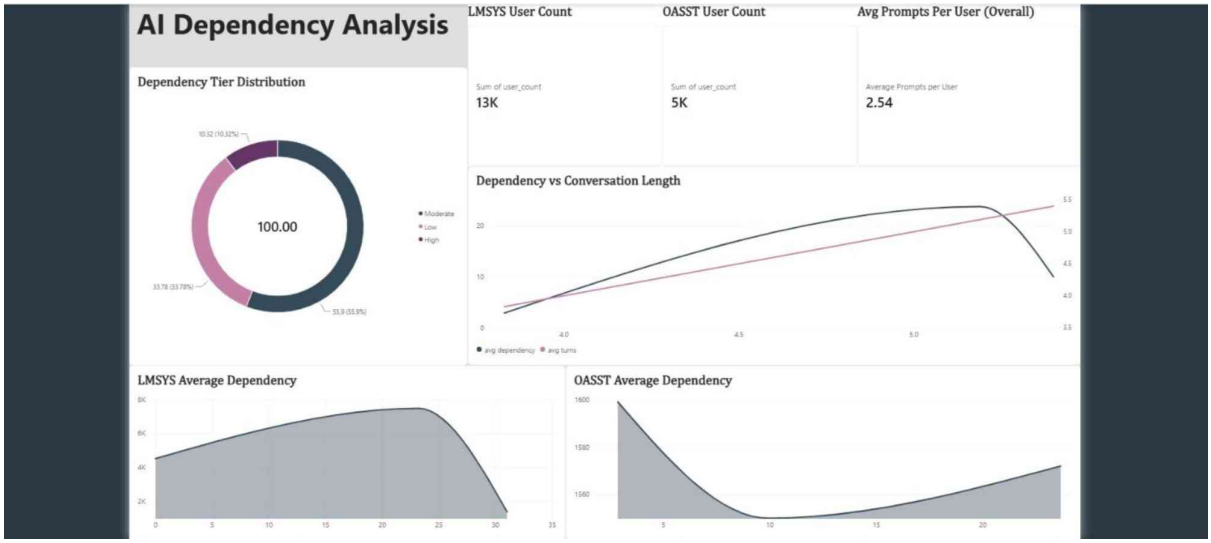


Figure 5: AI dependency dashboard.

- Population: about 13K LMSYS and 5K OASST users.
- Mean dependency: LMSYS 18.10, OASST 12.16 (0–100). These are not comparable; components, weights and normalisation differ.
- Both means are low because volume is heavy-tailed: a few very active users stretch the scale.
- Dependency rises with average conversation length over most of the range, then drops at the longest conversations.
- Average turns is a score component (25–30% weight), so part of that rise is by construction.
- *Not used:* Tier donut (not equal thirds); LMSYS and OASST area charts (summed, not averaged); the 2.54 prompts-per-user tile (divides by students and ChatGPT with no users).

8.5 Student AI Usage Dashboard

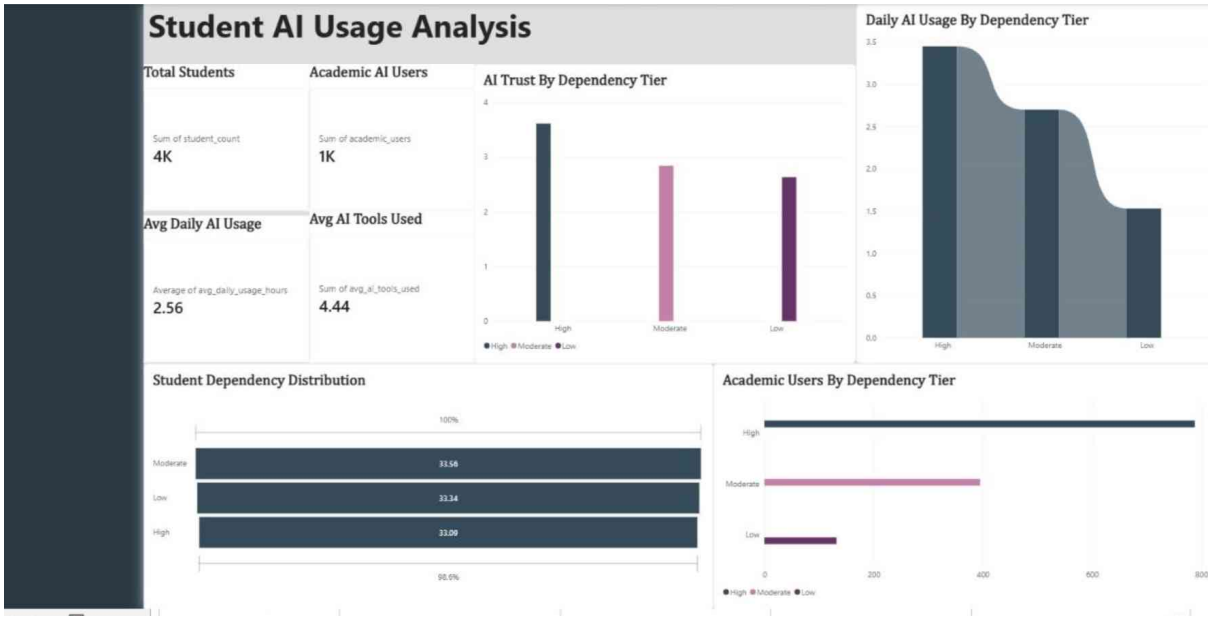


Figure 6: Student AI usage dashboard.

- 3,614 students; mean daily AI use 2.56 hours.
- Tiers are near equal thirds (33.6%, 33.3%, 33.1%), as expected from tercile bucketing.
- Daily use by tier (approx.): High 3.5 h, Moderate 2.7 h, Low 1.5 h, so High is over twice Low.
- Trust by tier (approx.): High 3.6, Moderate 2.9, Low 2.7.
- Academic users by tier (approx.): High 790, Moderate 400, Low 130.
- Caveat: hours, trust and academic use are score components, so these gradients are largely by construction.
- *Not used:* Avg AI Tools Used tile ("Sum of avg").

8.6 Recommendations Dashboard

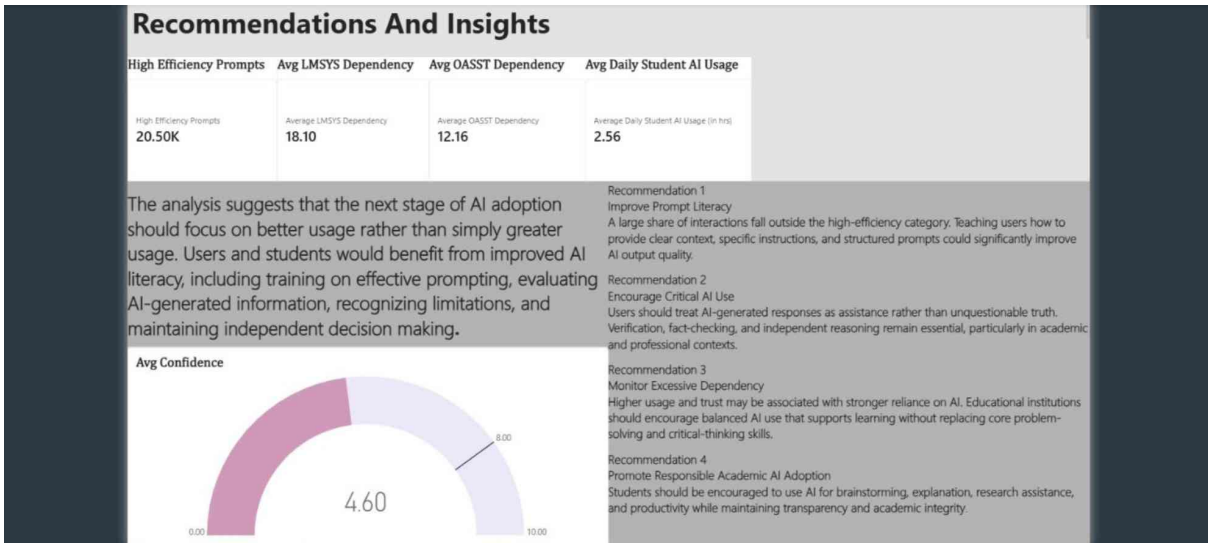


Figure 7: Recommendations dashboard.

- Improve prompt literacy: strongly supported by 62.9% of prompts not being High.
- Encourage critical use of AI output: sensible, but not tested by this data.
- Monitor excessive dependency: weaker support, since dependency is a constructed index and the tier gradients are partly circular.
- Promote responsible academic adoption: policy advice; the data shows usage levels, not outcomes.
- *Not used:* Avg Confidence gauge (4.60 on a 0–10 scale, against 0.59 elsewhere).

9 Limitations

1. **Labels are a proxy.** Efficiency is defined behaviourally (longer prompts, fewer follow-ups, no repeats). It has not been validated against human judgement or answer quality, and a short prompt can be perfectly efficient.
2. **Indirect length leakage.** Text features such as unique-word ratio and punctuation density are mechanically tied to length; Table 5 quantifies the effect but cannot remove it.
3. **Tree size is not only the user's doing.** OASST tree size reflects how the collection process built conversations.
4. **Domain shift.** The model is trained on OASST and applied to LMSYS and ChatGPT prompts, which differ structurally; the ChatGPT set has only 153 rows.
5. **Forced tertiles.** Labels and tiers are equal-sized by construction, so tier counts do not measure how many users are truly highly dependent.
6. **Dependency is a constructed index.** Weights are judgemental, and low-efficiency rate serves as both a quality measure and a dependency input.
7. **Population bias.** OASST contributors, LMSYS judges and surveyed students are not representative of general AI users, and the survey is self-reported.

10 Future Work

Validate efficiency labels against a human-annotated sample; add embedding-based features; replace tertile cut-offs with absolute thresholds; calibrate predicted probabilities per source; use outcome signals such as LMSYS winner or OASST rank; and add drift monitoring with scheduled retraining.

11 Conclusion

The project shows how heterogeneous AI-usage data can enter one analytical framework without false equivalences: one text-only classifier scores prompts everywhere, while dependency is measured separately within each population using only the signals it has. Its strengths are the leakage-aware design, the refusal to fabricate identity links, and a deployment-ready batch job. The main open task is external validation of what the efficiency and dependency scores actually measure.

A Reproducibility

Seed	42
Model	`RandomForestClassifier(n_estimators=300, max_depth=10, class_weight="balanced")`
Training & analysis	`ai_usage_patterns_analysis_1.py`
Batch scoring	`inference.py`
Experiments	`experiments/run_experiments.py`
Build paper	`cd paper && pdflatex whitepaper && pdflatex whitepaper`

B Athena Tables

`unified_prompt_efficiency`	prompt_key, source, text, predicted_label, confidence_score
`oasst_user_dependency`	user_id, prompt_volume, repeat_rate, avg_turns, low_efficiency_rate, dependency_score, dependency_tier
`lmsys_user_dependency`	judge_user_id, prompt_volume, avg_turns, repeat_rate (null), low_efficiency_rate, dependency_score, dependency_tier
`survey_dependency`	student_name, daily_usage_hours, trust_in_ai_tools, usage_bucket, academic_use_flag, num_ai_tools_used, dependency_score, dependency_tier